I. Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset:

A. Data type of all columns in the "customers" table.

Ans:

```
select

column_name,

 data_type

from Target.INFORMATION_SCHEMA.COLUMNS
where table_name = "customers"
```

Processing location: US

Query results

Save results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	column_name	data_type
1	customer_id	STRING
2	customer_unique_id	STRING
3	customer_zip_code_prefix	INT64
4	customer_city	STRING
5	customer_state	STRING

Results per page: 50 1 – 5 of 5

Insight: Almost all the columns in the table have the data type `STRING`, except for `zipcode`, which is `INT64`. Since `customer_id` is stored as a `STRING`, we can infer that it likely contains alphanumeric values.

B. Get the time range between which the orders were placed.

Ans:

```
select

      min(order_purchase_timestamp) as first_order,
      max (order_purchase_timestamp) as last_order
from `Target.orders`
```

Query results

Save results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	first_order	last_order
1	2016-09-04 21:15:19 UTC	2018-10-17 17:30:18 UTC

Results per page: 50 1 – 1 of 1

Insight: As we can see from retrieving the details of the first and last orders placed in our data, the range of our dataset spans approximately 2 years, from September 2016 to October 2018.

C. Count the Cities & States of customers who ordered the given period.

Ans:

```
select count(distinct customer_city) as count_of_unique_city,

count(distinct customer_state) as count_of_unique_state
from Target.customers
```

Query results

Save results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	count_of_unique_...	count_of_unique_...
1	4119	27

Results per page: 50 1 – 1 of 1

Insights: As we can see from the given timeframe, customers placed orders from 4,119 different cities across Brazil and from all 27 states. Since Brazil has a total of 27 states, this indicates that our customers come from all over the country

2.----In-depth Exploration:

A. Is there a growing trend in the no. of orders placed over the past years?

Ans:

```
with year_sales as (

select distinct(extract(year from order_purchase_timestamp)) as year,count(*) as total_orders

from Target.orders

group by year
order by year
)

select *,
       lag(total_orders) over (order by year)  as prev_year,
       round(case
           when lag(total_orders) over (order by year) =0 or lag(total_orders) over (order by
year) is null then null
           else ((total_orders)-lag(total_orders) over (order by year))*100/lag(total_orders)
over (order by year)
           end,2)as growth_over_year
from year_sales
order by year
```

Query results

Save results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	year	total_orders	prev_year	growth_over_year
1	2016	329	null	null
2	2017	45101	329	13608.51
3	2018	54011	45101	19.76

Results per page: 50 1 – 3 of 3

Insight: Since our data starts from September 2016, the order count that year (329) is too small to give meaningful insights, which also inflates the 2017 growth rate (13k%). The more reliable comparison is 2017 to 2018, where orders grew by 19.76%. This clearly shows a year-over-year increase in orders.

B. Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

Ans:

```
select * ,count(*) as Number_of_order

From

(select FORMAT_DATETIME("%B", order_purchase_timestamp) as month,

        FORMAT_DATETIME("%Y", order_purchase_timestamp) as Year
from `Target.orders`) s
```

```
group by s.month,s.Year
order by month asc
```

Query results

Save results Open in

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	month	Year	Number_of_order
1	April	2017	2404
2	April	2018	6939
3	August	2017	4331
4	August	2018	6512
5	December	2017	5673
6	December	2016	1
7	February	2017	1780
8	February	2018	6728
9	January	2017	800
10	January	2018	7269
11	July	2017	4026

Results per page: 50 1 – 25 of 25

Insight: Since our data starts at the end of 2016 with very few sales, we will focus only on 2017 and 2018. At first glance, comparing all months shows a significant increase in orders from 2017 to 2018. Considering seasonality, both years indicate stronger sales in May, June, July, and August, suggesting that business performs better during autumn and winter.

C.During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)
0-6 hrs : Dawn
7-12 hrs : Mornings
13-18 hrs : Afternoon
19-23 hrs : Night

Ans:
```
select *,count(*) as count_ord_placed

from
(select case
            WHEN extract(hour from order_purchase_timestamp) between 0 and 6 then "Dawn"
            WHEN extract(hour from order_purchase_timestamp) between 7 and 12 then "Mornings"
            WHEN extract(hour from order_purchase_timestamp) between 13 and 18 then
"Afternoon"
            WHEN extract(hour from order_purchase_timestamp) between 19 and 23 then "Night"
            end as time_window
from `Target.orders`) d
group by time_window
```

Query results

Save results Open in

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	time_window	count_ord_placed
1	Night	28331
2	Dawn	5242
3	Afternoon	38135
4	Mornings	27733

Insight: From analyzing the data across different times of the day (night, dawn, morning, and afternoon), we can see that night and morning have almost the same order counts (28,331 and 27,733), dawn has the lowest (5,242), and afternoon has the highest (38,135).

3.---Evolution of E-commerce orders in the Brazil region:

A.Get the month on month no. of orders placed in each state.

Ans:
```
select  c.customer_state,

        format_datetime("%B",o.order_purchase_timestamp) as months,
```

```
      count(o.order_purchase_timestamp) as count_order
from `Target.customers` c
join `Target.orders` o on c.customer_id = o.customer_id
group by c.customer_state,format_datetime("%B",o.order_purchase_timestamp)
order by count_order desc
```

Query results

Save results | Open in

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	customer_state	months	count_order
16	RJ	July	1288
17	MG	March	1237
18	MG	May	1190
19	MG	August	1177
20	RJ	February	1176
21	RJ	April	1172
22	RJ	June	1128
23	MG	July	1111
24	MG	June	1080
25	MG	February	1063
26	MG	April	1061

Results per page: 50 1 – 50 of 322

Insights: We analyzed orders by state and month, but since results from different years merged into the same month for each state, going into further granularity would not add meaningful insights. Therefore, we aggregated all years by month, allowing us to identify the best-performing months across years.

B. How are the customers distributed across all the states?

Ans:

```
select customer_state,
      count(*) as count_cust,
       round(count(*) * 100.0 / sum(count(*)) over (), 2) AS percent_contri
from `Target.customers`
group by customer_state
order by  count_cust desc
```

Query results

Save results | Open in

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	customer_state	count_cust	percent_contri
1	SP	41746	41.98
2	RJ	12852	12.92
3	MG	11635	11.7
4	RS	5466	5.5
5	PR	5045	5.07
6	SC	3637	3.66
7	BA	3380	3.4
8	DF	2140	2.15
9	ES	2033	2.04
10	GO	2020	2.03
11	PE	1652	1.66

Results per page: 50 1 – 27 of 27

Insights: From our analysis, we can see the customer distribution by state. São Paulo contributes the most with about 41% of customers, followed by Rio de Janeiro at 12.92% and Minas Gerais at 11.7%. This highlights both our top- and lowest-performing states, allowing us to design offers or strategies to boost customers in underperforming regions or identify potential issues there.

4.) ----Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight and others.

A. Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only). *You can use the "payment_value" column in the payments table to get the cost of orders.*

Ans: `with sub as`

```
(
      select
       extract(year from o.order_purchase_timestamp) as year,
       extract (month from o.order_purchase_timestamp) as month_num,
       format_datetime('%B' , o.order_purchase_timestamp) as month,
       sum(p.payment_value) as total_payment
from `Target.orders` o
join  `Target.payments` p on  o.order_id = p.order_id
group by extract(year from o.order_purchase_timestamp),format_datetime('%B' ,
o.order_purchase_timestamp),extract (month from o.order_purchase_timestamp)
),
sep as
(select t1.year as prev_2017,
        t1.month_num as prev_mon,
        t1.month as prev_month_name,
        t1.total_payment as prev_total,
        t2.year,
        t2.month_num,
        t2.month,
        t2.total_payment
from
(select * from sub
where year = 2017 and month_num between 1 and 8
order by month) t1
join
(select * from sub
where year = 2018 and month_num between 1 and 8
order by month)t2 on t1.month_num = t2.month_num
)


select sep.month ,
      round((sep.total_payment - sep.prev_total)*100/sep.prev_total,2) as month_on_month
 from sep
 order by sep.month_num
```

Query results

Save results Open in

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	month	month_on_month
1	January	705.13
2	February	239.99
3	March	157.78
4	April	177.84
5	May	94.63
6	June	100.26
7	July	80.04
8	August	51.61

Results per page: 50 1 – 8 of 8

Insights: From the analysis, we can see that comparing January 2017 to January 2018 shows a 700% increase in order cost, while August recorded the lowest growth.

B. Calculate the Total & Average value of order price for each state.

Ans:

```
select c.customer_state,

      round(sum(p.payment_value),2) as total_amount,
      round(avg(p.payment_value),2) as avg_total
from `Target.customers`  c
join `Target.orders`  o on  c.customer_id = o.customer_id
join `Target.payments` p on o.order_id = p.order_id
group by c.customer_state
```

Query results

Save results Open in

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	customer_state	total_amount	avg_total
1	MG	1872257.26	154.71
2	SP	5998226.96	137.5
3	CE	279464.03	199.9
4	RS	890898.54	157.18
5	RJ	2144379.69	158.53
6	MT	187029.29	195.23
7	GO	350092.31	165.76
8	SC	623086.43	165.98
9	PR	811156.38	154.15
10	ES	325967.55	154.71

Results per page: 50 1 – 27 of 27

Insights: From the analysis of total order value and average order value by state, we can see that states with a high total order value tend to have a lower average purchase, while states with a low total order value tend to have a higher average purchase.

C. Calculate the Total & Average value of order freight for each state.

Ans:

```
select c.customer_state,
       round(sum(freight_value),2) as total_freight,
       round(avg(freight_value),2) as avg_freight
from `Target.customers` c
join `Target.orders` o on c.customer_id = o.customer_id
join `Target.order_items` ot on ot.order_id = o.order_id
group by c.customer_state
order by round(sum(freight_value),2) desc,round(avg(freight_value),2) desc
```

Query results

Save results Open in

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	customer_state	total_freight	avg_freight
1	SP	718723.07	15.15
2	RJ	305589.31	20.96
3	MG	270853.46	20.63
4	RS	135522.74	21.74
5	PR	117851.68	20.53
6	BA	100156.68	26.36
7	SC	89660.26	21.47
8	PE	59449.66	32.92
9	GO	53114.98	22.77
10	DF	50625.5	21.04
11	ES	49764.6	22.06

Results per page: 50 1 – 27 of 27

Insights: From the analysis of freight costs by state (total and average), we can see that states with high total freight costs tend to have a lower average freight cost, and vice versa. The main reason could be better connectivity in some states, while lower order volumes in others may also drive up average freight costs.

5).
----Analysis based on sales, freight and delivery time.

A). Find the no. of days taken to deliver each order from the order's purchase date as delivery time.

Also, calculate the difference (in days) between the estimated & actual delivery date of an order.
Do this in a single query.
You can calculate the delivery time and the difference between the estimated & actual delivery date using the given formula:
time_to_deliver = order_delivered_customer_date - order_purchase_timestamp
diff_estimated_delivery = order_delivered_customer_date - order_estimated_delivery_date

Ans:

```
select order_id,
       date_diff(order_delivered_customer_date, order_purchase_timestamp, day) as
       time_to_deliver,
```

```
date_diff(order_delivered_customer_date,order_estimated_delivery_date,day) as
diff_estimated_deliver
from `Target.orders`
```

Query results

Save results | Open in

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	order_id	time_to_deliver	diff_estimated_d...
1	65d1e226dfaeb8cdc42f665422...	35	-16
2	2c45c33d2f9cb8ff8b1c86cc28...	30	-28
3	1950d777989f6a877539f53795...	30	12
4	bfbd0f9bdef84302105ad712db...	54	36
5	98974b076b01553d49ee64679...	43	-6
6	c4b41c36dd589e901f6879f25a...	36	-14
7	d2292ff2201e74c5db154d1b7a...	29	-20
8	95e01270fcbae986342340010...	30	-19
9	ed8c7b1b3eb256c70ce0c7423...	44	-5

Results per page: 50 1 – 50 of 99441

Insights: Most orders have a negative **`diff_estimated_delivery`**, meaning they were delivered earlier than estimated. On average, it takes 12 days to deliver orders, and the average `diff_estimated_delivery` is -10, suggesting that most deliveries are completed about 10 days earlier than the estimated date.

B). *Find out the top 5 states with the highest & lowest average freight value.*

Ans.

```
with avg_fg as (

select c.customer_state,
      avg(ot.freight_value) as avg_freight
from `Target.customers` c
join `Target.orders` o on c.customer_id = o.customer_id
join `Target.order_items` ot on ot.order_id = o.order_id
group by c.customer_state
)

(select *
from avg_fg
order by avg_fg.avg_freight desc
limit 5)

union all
(select *
from avg_fg
order by avg_fg.avg_freight asc
limit 5
)
```

Query results

Save results | Open in

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	customer_state	avg_freight
1	RR	42.98442307692...
2	PB	42.72380398671...
3	RO	41.06971223021...
4	AC	40.07336956521...
5	PI	39.14797047970...
6	SP	15.14727539041...
7	PR	20.53165156794...
8	MG	20.63016680630...
9	RJ	20.96092393168...
10	DF	21.04135494596...

Insights: From our analysis, we identified the 5 highest and 5 lowest average freight costs. The highest is 45.98, followed by the next values, while the lowest is 15.14, followed by the others.

C.Find out the top 5 states with the highest & lowest average delivery time.

Ans:
```
with state_avg as (
select c.customer_state,
       round(avg(date_diff(o.order_delivered_customer_date, o.order_purchase_timestamp,
day)),2) as time_to_deliver
from `Target.orders` o
join `Target.customers` c on  o.customer_id = c.customer_id
group by c.customer_state
)

(select *
from state_avg
order by state_avg.time_to_deliver desc
limit 5)
union all
(select *
from state_avg
order by state_avg.time_to_deliver
limit 5)
```

Query results

Save results | Open in

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	customer_state	time_to_deliver
1	RR	28.98
2	AP	26.73
3	AM	25.99
4	AL	24.04
5	PA	23.32
6	SP	8.3
7	PR	11.53
8	MG	11.54
9	DF	12.51
10	SC	14.48

Insights: Some states have significantly longer delivery times (e.g., remote or less connected states), which may indicate logistics or connectivity issues. States with the lowest average delivery times are likely closer to main distribution hubs, showing efficient supply chain coverage

D). Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.

Ans:
```
select *
from (
select c.customer_state,
      round(avg(date_diff(o.order_delivered_customer_date,o.
      order_estimated_delivery_date,day)),2) delivery_time

from `Target.orders` o
join `Target.customers` c on o.customer_id = c.customer_id
group by c.customer_state)d
order by d.delivery_time
limit 5
```

Query results

Save results | Open in

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	customer_state	delivery_time
1	AC	-19.76
2	RO	-19.13
3	AP	-18.73
4	AM	-18.61
5	RR	-16.41

Results per page: 50 | 1 – 5 of 5

Insights: These states have significantly negative values, meaning orders are being delivered on average 16–20 days earlier than the estimated date**.** Acre (AC) shows the fastest deliveries, averaging ~20 days early, followed closely by Rondônia (RO) and Amapá (AP).

6). -----Analysis based on the payments:

A). Find the month on month no. of orders placed using different payment types.

Ans:

```
select p.payment_type,
      extract(month from o.order_purchase_timestamp) as month,
      extract(year from o.order_purchase_timestamp) as year,
      count(distinct o.order_id) as number_order
from `Target.orders` o
join `Target.payments` p on o.order_id = p.order_id
group by 1,2,3
order by 3,2
```

Query results

Save results | Open in

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	payment_type	month	year	number_order
1	credit_card	9	2016	3
2	voucher	10	2016	11
3	credit_card	10	2016	253
4	UPI	10	2016	63
5	debit_card	10	2016	2
6	credit_card	12	2016	1
7	credit_card	1	2017	582
8	UPI	1	2017	197
9	debit_card	1	2017	9
10	voucher	1	2017	33

Results per page: 50 1 – 50 of 90

Insights: From the analysis, credit cards are the most used payment method each month, growing in line with sales. UPI is the second most used, while debit cards are the least.

B. Find the no. of orders placed on the basis of the payment installments that have been paid.

Ans:

```
select payment_installments,
       count(distinct order_id) as num_orders
from `Target.payments`
where payment_sequential = payment_installments
group by 1
order by 1;
```

Query results

Save results | Open in

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	payment_installm...	num_orders
1	1	48236
2	2	53
3	3	1

Insights: Most customers (48,236 orders) pay in a **s**ingle installment, showing a strong preference for upfront/full payments rather than EMIs.